

Artificial Intelligence Requirement Analyzer



Developed By

Laiba Arshad	UOC-CS-F2022-010
Alishba Rustam	UOC-CS-F2022-029
Nayab Nazir	UOC-CS-F2022-040

Supervised By

Mr. Wasi Haider

Department of Computer Science & Information Technology

Faculty of Computing & Information Technology

The University of Chakwal

Bachelor of Science in Computer Science / Information

Technology (2022-26)

DECLARATION

We hereby declare that this project titled “**Artificial Intelligence Requirement Analyzer (AIRA)**” is our own work and has not been copied from any source. It is further declared that we have developed this software documentation and report entirely based on our personal efforts. If any part of this project is found copied or reproduced from another source, we will be responsible for the consequences. No portion of this work has been submitted previously for any degree.

Alishba Rustam

Laiba Arshad

Nayab Nazir

CERTIFICATE OF APPROVAL

It is to certify that the final year project of “**Artificial Intelligence Requirement Analyzer (AIRA)**” has successfully been developed and submitted by **Laiba Arshad (UOC-CS-F2022-010)**, **Alishba Rustam (UOC-CS-F2022-029)** and **Nayab Nazir (UOC-CS-F2022-040)** under the supervision of **Mr. Wasi Haider** at the Department of Computer Science & Information Technology, The University of Chakwal and that in his opinion, it is fully adequate in scope, quality, and fulfills the requirements for the degree of Bachelor of Science in Computer Science / Information Technology, Session 2022–26.

Mr. Wasi Haider
Project Supervisor

Dr./Mr./Ms. _____
External Examiner

Ms. Hina Iftikhar
Head of Department

ABSTRACT

Software documentation is an essential activity in software engineering because it records requirements, design decisions, and expected system behavior. The current implementation of Artificial Intelligence Requirement Analyzer (AIRA) is a web-based documentation assistant that supports Software Requirements Specification (SRS) generation, SRS ambiguity analysis, UML diagram generation, UML image explanation, editable outputs, local export, and user history.

The system is implemented with HTML, CSS, and JavaScript on the frontend; Node.js and Express.js on the backend; MySQL for persistence; and Python services for machine-learning and OCR tasks. The backend exposes API routes for authentication, file extraction, SRS generation, ambiguity checking, UML generation, UML image description, history retrieval, and PlantUML rendering.

The AI layer uses code-supported techniques only: TF-IDF based retrieval, scikit-learn pipelines, Logistic Regression for SRS ambiguity classification, LinearSVC for UML diagram type prediction, rule-based requirement checks, OpenCV/Pillow image processing, and Tesseract OCR. Optional Gemini and OpenRouter providers are used only when valid API keys are configured.

This report has been revised according to the current source code. Claims that are not evidenced by the implementation have been removed or treated as limitations/future work. AIRA remains an assistance tool whose generated outputs require human review before academic or professional submission.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge, by virtue of which we were able to accomplish this challenging task of developing the “**Artificial Intelligence Requirement Analyzer (AIRA)**”.

We are greatly indebted to our project supervisor **Mr. Wasi Haider** for his continuous guidance, valuable suggestions, and personal supervision throughout the completion of this project. His support and encouragement played a vital role in successfully achieving our project objectives.

We are also thankful to our parents and family members who have been a constant source of motivation and encouragement and have instilled in us the values of honesty and hard work.

Alishba Rustam

Laiba Arshad

Nayab Nazir

PREFACE

This final year project report has been prepared in fulfillment of the requirements for the Bachelor of Science in Computer Science / Information Technology, session 2022-26, in the academic year 2026.

The project titled Artificial Intelligence Requirement Analyzer (AIRA) focuses on assisting users in preparing SRS and UML documentation through a web interface connected with backend APIs, a MySQL database, and Python-based AI services.

During development, the implemented tools and technologies include HTML, CSS, JavaScript, Node.js, Express.js, MySQL, Python, scikit-learn, joblib, OpenCV, Pillow, Tesseract OCR, PDF.js, PlantUML, and optional Gemini/OpenRouter integration.

The project was developed through planning, design, implementation, testing, and revision phases. The revised documentation is based on source-code evidence rather than unsupported assumptions.

This project falls under Artificial Intelligence, Web Application Development, Requirements Engineering, UML Modeling, Natural Language Processing, and OCR-based image interpretation.

We are very thankful to our project supervisor “**Mr. Wasi Haider**” who has helped us to complete this project and we really appreciate all his efforts and support.

We are also very thankful to The University of Chakwal and all the teachers & staff.

ABBREVIATIONS

The following abbreviations are used in the implemented AIRA project.

Abbreviation	Meaning
AI	Artificial Intelligence
AIRA	Artificial Intelligence Requirement Analyzer
API	Application Programming Interface
DBMS	Database Management System
ERD	Entity Relationship Diagram
HTML	Hypertext Markup Language
JSON	JavaScript Object Notation
ML	Machine Learning
OCR	Optical Character Recognition
SRS	Software Requirements Specification
TF-IDF	Term Frequency-Inverse Document Frequency
UML	Unified Modeling Language
UI	User Interface

Table of Contents

Chapter 1: Introduction of the Project	1
1.1. Introduction:	1
1.2. Scope:	1
1.3. Purpose:	2
1.4. Objective:.....	2
1.5. Tools and Technologies:.....	3
1.6. Chapter Summary	5
Chapter 2: Literature Review & Existing Systems.....	6
2.1. Literature Review:	6
2.2. Problem Statement:.....	7
2.3. Proposed Solution:.....	7
2.4. Comparative Analysis between Existing and Proposed Solution:.....	8
2.5. Machine Learning Techniques / Algorithms:	9
2.6 Chapter Summary	11
Chapter 3: Software Requirements Specification.....	12
3.1. Functional Requirements:.....	12
3.2. Non-Functional Requirements:.....	13
3.3. Use Case Diagram:	13
3.4. Methodology and Software Lifecycle for This Project	16
3.5. Chapter Summary	17
Chapter 4: Design and Architecture	18
4.1. Entity Relationship Diagram (ERD):.....	18
4.2. Sequence Diagrams:	18
4.3. Architecture Design Diagram:	18
4.4. Class Diagram:.....	18
4.5. Database Design:	18
4.6. Interface Design:.....	18
4.7. Summary.....	21
Chapter 5: Software Testing & Implementation	22
5.1 Deriving Test:	22
5.2 Test Environment:	22
5.3 Testing Techniques:.....	23
5.4 Test Cases:	24

5.5	Chapter Summary:	25
Chapter 6:	Conclusion and Future Work.....	27
6.1	Discussion:.....	27
6.2	Conclusion:	27
6.3	Limitations:.....	28
6.4	Future Work:.....	28
	References.....	29

List of Figures/Diagrams

Figure 1: Use Case of System.....	13
Figure 2: SRS Generation.....	14
Figure 3 SRS Analysis.....	14
Figure 4: UML Generation	15
Figure 5: Software Lifecycle	16
Figure 6: Entity Relationship Diagram.....	18
Figure 7: Sequence Diagram	18
Figure 8: Architecture Design Diagram	18
Figure 9: Class Diagram	18
Figure 10: Database Design Diagram.....	18
Figure 11: Sign Up Screen.....	18
Figure 12: home Screen	19
Figure 13: Screen SRS.....	19
Figure 14: UML Screen Diagram	19
Figure 15: UML Image Description Screen	20
Figure 16: My History Screen	20

List of Tables

Table 4-1 Comparative Analysis between Existing and Proposed Solution	8
Table 4-1 User Login.....	24
Table 4-2 SRS Generation	24
Table 4-3 UML Generation	25
Table 4-4 SRS Ambiguity Analysis	25

Chapter 1: Introduction of the Project

1.1. Introduction:

Software documentation plays an important role in the success of a software project. Requirements specifications and UML diagrams help define what a system should do, the constraints it must satisfy, and how different components interact. Preparing and analyzing these artifacts manually can be time-consuming, especially when requirements are incomplete, ambiguous, or scattered across multiple sources.

AIRA (Artificial Intelligence Requirement Analyzer) is a web-based software documentation assistance system that helps users create and analyze software requirements and design artifacts. **The system allows users to enter a project idea or upload supporting files, generate structured SRS documents, detect requirement ambiguity, generate UML diagrams, describe uploaded UML diagram images, edit generated outputs, download results, and reuse previous work through a history feature.**

The project reduces manual effort by integrating a browser-based interface, HTML/CSS/JavaScript frontend pages, Express.js APIs, MySQL database storage, Python AI services, and automated document-processing capabilities into a single documentation workspace. The architecture separates user interaction, API orchestration, data persistence, and AI processing, providing an organized environment for software requirements and design documentation.

1.2. Scope:

The implemented scope includes:

- SRS generation from project title, project details, selected features, uploaded documents, and extracted UML image text.
- SRS ambiguity analysis using trained ML and rule-based checks.
- UML generation from textual system descriptions or extracted document content.
- UML diagram types: use case, class, sequence, ERD, and activity.
- UML image description from uploaded images, editable AIRA diagram projects, or user-confirmed diagram structures.
- User signup, login, role decoration, and access-tier enforcement.

- MySQL storage for users, projects, uploaded files, generated outputs, analysis reports, diagram outputs, image descriptions, exports, and activity history.
- Frontend settings, multilingual UI preferences, editable outputs, and browser-based download facilities.

The implemented scope excludes features not present in the source code, including real-time multi-user collaboration, mobile application support, voice input, a complete admin dashboard, and a production deployment pipeline.

1.3. **Purpose:**

The purpose of this project is to:

- To reduce the effort required to create initial Software Requirement Specification (SRS) documents and UML artifacts.
- To assist users in generating structured software requirement documents from project ideas and supporting files.
- To identify ambiguous, vague, or unclear requirement statements in SRS documents.
- To generate UML outputs that support software design and system modeling activities.
- To describe and interpret uploaded UML diagram images for better understanding and documentation.
- To provide editing capabilities for generated SRS and UML outputs.
- To enable users to download generated documentation and analysis results.
- To maintain a history of generated work, allowing users to revisit and reuse previous prompts and outputs.
- To improve the reviewability and consistency of software requirements and design documentation.
- To provide an integrated environment for software documentation generation and analysis.

1.4. **Objective:**

The project objectives are:

- To implement a web interface for SRS generation, UML generation, SRS ambiguity analysis, UML image description, account access, history, settings, and downloads.
- To implement an Express API layer that validates requests, calls AI services, persists generated outputs, and returns structured JSON responses.

- To integrate Python AI services for requirement retrieval, ambiguity detection, UML generation, OCR, and optional cloud AI processing.
- To design and use a MySQL schema that stores users, projects, uploaded files, generated SRS documents, SRS analysis reports, UML requests, UML outputs, UML image descriptions, export records, activity history, and usage events.
- To provide editable output previews and local export options for generated documents and diagrams.
- To document current limitations and mark deployment-dependent behavior for manual verification.

1.5. Tools and Technologies:

The implemented tools and technologies identified from the current source code are listed below.

1.5.1. Frontend:

- HTML5
- CSS3
- JavaScript
- Font Awesome icons
- Browser localStorage
- Blob-based file downloads
- File System Access API (where supported)

1.5.2. Backend:

- Node.js
- Express.js
- API routes
- JSON request parsing
- Node.js child processes
- Python AI and OCR services invoked from the backend

1.5.3. Database:

- MySQL database (aira_db)
- mysql2 connection pool
- SQL schema files

1.5.4. AI and Machine Learning:

- Python
- scikit-learn
- joblib
- TF-IDF vectorization
- Cosine similarity analysis
- Logistic Regression
- LinearSVC
- Rule-based ambiguity detection

1.5.5. OCR and Vision Processing:

- OpenCV
- Tesseract OCR
- Pillow (PIL)

1.5.6. Document and File Processing:

- DOCX XML extraction
- PDF.js
- Custom PDF text extraction fallback
- Image OCR processing

1.5.7. Optional External AI Services:

- Gemini API
- OpenRouter API
- Environment-based configuration through .env files

1.5.8. UML Rendering and Export:

- PlantUML source generation
- PlantUML rendering endpoint
- PNG export
- PDF export
- SVG export
- Text export

1.5.9. Development Environment:

- Windows-based local development setup
- Visual Studio Code (VS Code)
- Node.js runtime
- Python environment

- MySQL Server
- XAMPP (where applicable)

1.5.10. System Requirements:

- Operating System: Windows 10 or Windows 11
- Processor: Intel Core i3 or above
- Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox
- Development Tools: VS Code, Jupyter Notebook, Node.js, MySQL, Python, and browser developer tools
- Internet connection required only for optional cloud AI providers and external PlantUML/Kroki rendering services

1.6. Chapter Summary

This chapter presented an overview of AIRA (Artificial Intelligence Requirement Analyzer) and established the foundation for the rest of the project report. It explained the problem domain of software requirements and design documentation and described how AIRA assists users in generating, analyzing, and managing SRS and UML artifacts. The chapter also defined the project scope, objectives, key functionalities, and target users.

In addition, the chapter outlined the overall technology stack used in the implementation, including the frontend, backend, database, AI services, and UML processing components. Particular emphasis was placed on aligning the documentation with the actual source code implementation to ensure accuracy and consistency. Unsupported assumptions and outdated descriptions from previous versions of the report were removed, and the system was described according to its current capabilities.

By providing a clear understanding of the project's purpose, architecture, and functional boundaries, this chapter established an implementation-driven basis for the remaining chapters, which discuss system analysis, design, implementation details, testing, and evaluation of the AIRA platform.

Chapter 2: Literature Review & Existing Systems

2.1. Literature Review:

Requirements engineering provides the foundation for SRS preparation. IEEE/ISO/IEC 29148-2018 describes requirements engineering across the system life cycle and defines quality characteristics for requirements [1]. AIRA's ambiguity analysis is aligned with this principle because it attempts to detect statements that are difficult to verify or interpret consistently.

UML is a standardized modeling language maintained by the Object Management Group. UML version 2.5.1 defines modeling constructs used to represent software structure and behavior [2]. AIRA implements generation and editing support for use case, class, sequence, ERD, and activity-oriented diagram outputs. The tool's PlantUML support follows the practical idea of representing UML diagrams as text that can be rendered to images [3].

The implemented machine-learning pipeline uses TF-IDF text representation, Logistic Regression, LinearSVC, and cosine-similarity retrieval. TF-IDF is a common method for transforming text into numerical features for machine learning [4]. Logistic Regression and SVM-family classifiers are standard supervised methods available in scikit-learn . The current system combines these trained pipelines with rule-based checks because ambiguity detection depends not only on statistical classification but also on requirement-quality indicators such as vague terms, optional wording, and missing measurable limits.

OCR and image-processing components support UML image description. Tesseract OCR provides text recognition, while OpenCV and Pillow provide image loading, enhancement, layout processing, and drawing capabilities [4], [5]. The system uses PDF.js for browser/server PDF text extraction support [12]. Optional Gemini and OpenRouter providers are configured for structured JSON generation and multimodal processing where API keys are available [4], [6].

2.2. Problem Statement:

Creating Software Requirements Specification (SRS) documents and UML diagrams is a fundamental activity in software engineering, but it is often time-consuming, inconsistent, and error-prone when performed manually. Students and beginner developers frequently struggle to express requirements clearly, resulting in vague, incomplete, or ambiguous statements that negatively impact system design, implementation, and testing.

Existing tools provide only partial support for this process. Word processors are used for writing documentation but lack automated requirement structuring and ambiguity detection. UML diagram editors enable visual modeling but require manual effort and prior knowledge of UML notation. Similarly, AI-based text generation tools can assist in drafting content but do not maintain structured project context, history, or integrated diagram handling within a unified workspace.

The core problem is the absence of a lightweight, integrated system that supports the complete software documentation workflow in a single environment. There is no dedicated platform for academic or small-scale projects that can generate structured SRS documents, analyze requirement ambiguity, create and interpret UML diagrams, support editing and revision, maintain project history, and provide organized export of documentation artifacts.

This fragmentation forces users to depend on multiple disconnected tools, increasing effort, reducing consistency, and making it difficult to maintain alignment between requirements and system design artifacts.

2.3. Proposed Solution:

To address this problem, AIRA is implemented as a browser-based application with an Express backend, a MySQL database, and Python AI services. The system currently:

- Generates SRS documents from user prompts, structured fields, uploaded documents, and extracted image text
- Creates Use Case, Class, Sequence, ERD, and Activity UML outputs from textual input
- Analyzes SRS statements for ambiguity using a trained Logistic Regression pipeline and rule-based checks

- Explains UML diagram images using OCR, image-quality validation, and optional cloud AI vision support

The system uses TF-IDF vectorization, cosine-similarity retrieval, Logistic Regression, LinearSVC, OpenCV, Pillow, Tesseract OCR, PlantUML rendering, and optional Gemini/OpenRouter APIs where keys are configured [4]-[10].

2.4. Comparative Analysis between Existing and Proposed Solution:

Feature area	Generic word processor	Generic diagram tool	Generic AI chat tool	Implemented AIRA system	References
SRS Generation	Manual	Not primary	Prompt based	Structured SRS generation with project context and uploaded files	[6]
Ambiguity checking	Manual review	Not supported	Prompt dependent	Rule and ML-based ambiguity report	[7]
UML generation	Manual text only	Manual diagram drawing	Prompt dependent	Use case, class, sequence, ERD, and activity outputs	[2]
UML image explanation	Not supported	Limited	Prompt dependent	OCR/vision-based description and confirmed-structure reporting	[3]
Editable diagram canvas	Not primary	Yes	No	Implemented with browser canvas/SVG-style workspace	[2]
User history	File based	File based	Account dependent	MySQL-backed activity history	[8]

Table 4-1 Comparative Analysis between Existing and Proposed Solution

2.5. Machine Learning Techniques / Algorithms:

The implemented project uses Artificial Intelligence techniques from text processing, machine learning, retrieval, OCR, and optional cloud generation. Unsupported techniques from the old report were removed because they are not present in the inspected source code.

2.5.1 Natural Language Processing (NLP)

NLP is used to process user prompts, uploaded document text, and requirement statements. In the current implementation, the main local representation is TF-IDF.

The following text-processing techniques are supported by the code:

TF-IDF Vectorization and Text Classification:

This technique converts requirement text into weighted numeric features for local scikit-learn models used in ambiguity analysis and UML diagram type prediction.

Rule-based Ambiguity Detection:

Rules identify vague terms, unclear quantities, optional behavior, subjective quality words, and compound statements that may require manual requirement review.

Cosine-Similarity Retrieval:

The SRS generation service retrieves similar examples from a stored TF-IDF index and uses them to support structured SRS drafting through similarity-based retrieval.

Cloud JSON Generation:

When Gemini or OpenRouter keys are configured, the system can request structured JSON output. If keys are missing, local fallback logic is used.

2.5.2. Machine Learning Algorithms

The codebase uses trained scikit-learn assets stored with joblib. Model behavior should be verified again before making final external accuracy claims.

Logistic Regression:

Logistic Regression is used in the SRS ambiguity pipeline. The stored metric report shows about 0.803 test accuracy and an ambiguous-class F1 score of about 0.468, so outputs need human review.

Linear Support Vector Classification:

LinearSVC is used for UML diagram type classification. The saved development report shows high test performance on a small dataset and should be manually verified before publication.

Nearest Pattern Retrieval:

A TF-IDF based retriever compares new prompts with stored diagram or requirement patterns and helps produce structured fallback outputs.

2.5.3. OCR and Image Processing

Image processing is used when users upload UML diagram images for explanation.

- The system checks image quality and uses OpenCV/Pillow processing before OCR where required.
- Tesseract OCR extracts visible text from UML images and supports later report generation.
- This helps users understand UML diagrams, but unclear images still require manual verification.

2.6 Chapter Summary

This chapter positions AIRA within requirements engineering, UML modeling, NLP, OCR, and AI-based documentation systems using established standards like IEEE/ISO/IEC 29148-2018 and UML 2.5.1. It shows how the system aligns with requirement quality principles and standardized software modeling practices.

The review highlights that existing tools handle SRS writing, UML design, and AI text generation separately, leading to fragmented workflows and manual effort. In contrast, AIRA integrates these functions into a single platform that supports structured SRS generation, UML diagram creation, ambiguity detection, OCR-based diagram interpretation, and project history management.

The chapter also clarifies the actual implemented techniques, including TF-IDF, Logistic Regression, LinearSVC, cosine-similarity retrieval, rule-based ambiguity detection, and OCR using Tesseract with OpenCV and Pillow. Optional cloud APIs are included only when configured. Unsupported methods such as NER, dependency parsing, and transformer embeddings were removed to maintain accuracy with the real codebase.

Overall, this chapter establishes the theoretical background, identifies gaps in existing systems, and justifies AIRA as an integrated solution for automated software documentation and modeling.

Chapter 3: Software Requirements Specification

3.1. Functional Requirements:

- The system shall allow users to create an account by providing full name, email address, and password.
- The system shall authenticate registered active users using email and password.
- The system shall hash user passwords using bcrypt before storing them in the database.
- The system shall assign user roles as free, premium, or admin based on database role and configured email lists.
- The system shall enforce daily usage limits for free-plan users on supported AI features.
- The system shall restrict non-English AI outputs and selected premium exports to premium and admin users only.
- The system shall generate SRS documents from project title, prompts, domain, feature list, actors, data notes, uploaded files, and extracted UML image text.
- The system shall extract text from supported file formats including TXT, DOCX, PDF, and image files.
- The system shall analyze SRS text and generate an ambiguity detection report.
- The system shall generate UML outputs from textual system descriptions or extracted document content.
- The system shall support UML generation for use case, class, sequence, ERD, and activity diagrams.
- The system shall render PlantUML source into SVG or PNG using a server-side rendering service.
- The system shall provide an editable diagram workspace with nodes and connectors.
- The system shall allow export of diagrams in PNG, PDF, SVG, text, and editable JSON (.airauml) formats based on user access level.
- The system shall describe uploaded UML diagram images using OCR and vision-based processing.
- The system shall convert confirmed UML structures into structured description reports.
- The system shall store generated SRS documents, ambiguity reports, UML outputs, and image descriptions in user history.
- The system shall allow users to view and reuse previous history records.

- The system shall allow users to edit generated SRS and analysis outputs before downloading.
- The system shall provide configurable settings including appearance, language, text size, start page, history saving, and export preferences.

3.2.Non-Functional Requirements:

- System should provide a clear and user-friendly web interface
- Backend routes should use timeout handling for long AI or rendering operations
- Passwords should be stored securely using bcrypt hashing
- AI outputs should be treated as drafts requiring manual verification
- Database tables should preserve traceability between users, projects, files, outputs, and history
- The application should run on common desktop browsers with the configured local backend

3.3. Use Case Diagram:

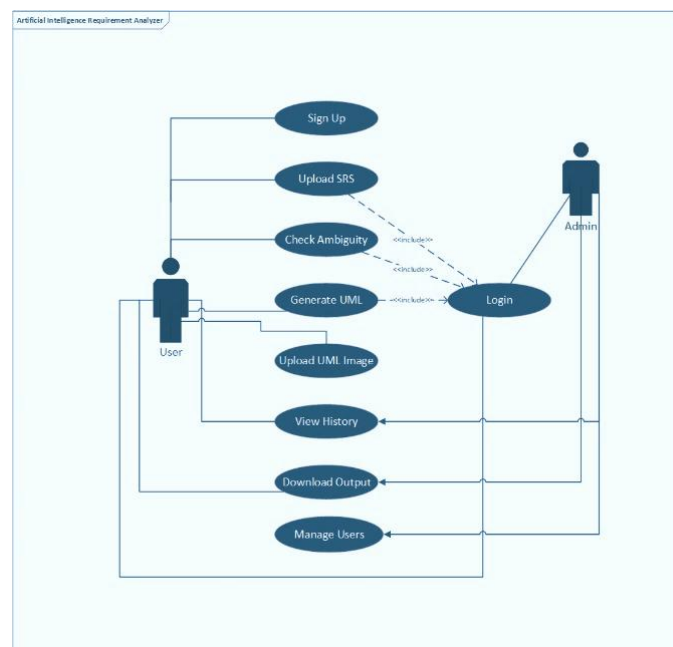


Figure 1: Use Case of System

3.3.1 SRS generation

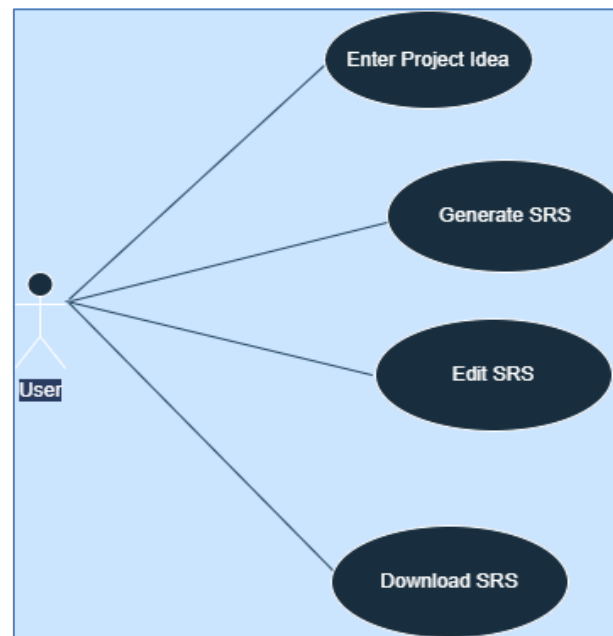


Figure 2: SRS Generation

3.3.2 SRS Analysis

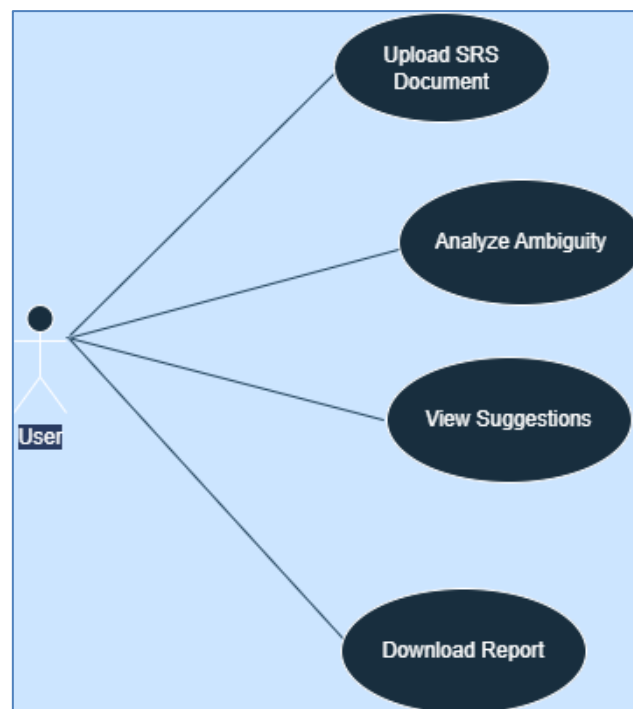


Figure 3 SRS Analysis

3.3.3 UML Generation

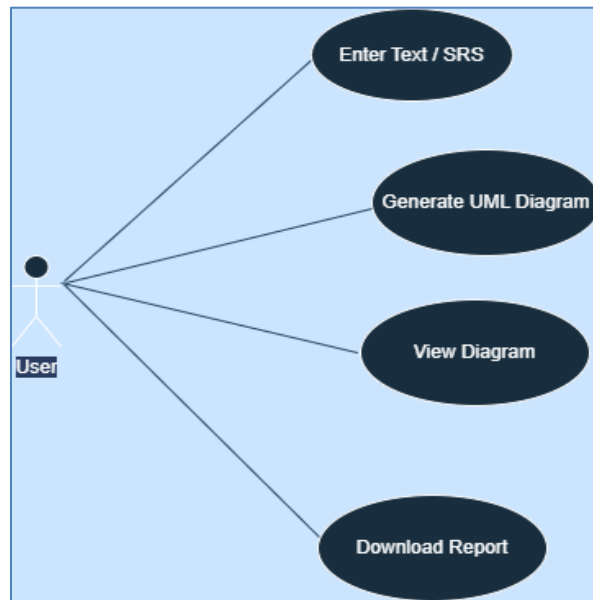


Figure 4: UML Generation

3.3.4 UML Image Explanation

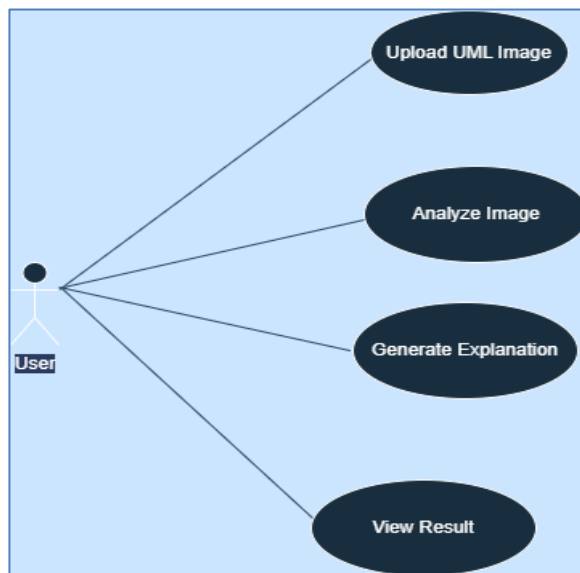


Figure 5: UML Image Explanation

3.4. Methodology and Software Lifecycle for This Project

The project follows an iterative development approach because:

- Requirements and documentation features changed during implementation
- AI services and model outputs required repeated refinement and fallback handling
- Frontend, backend, database, and Python modules had to be integrated step by step

An iterative approach allowed the team to improve SRS, UML, analysis, OCR, and history features according to testing feedback.

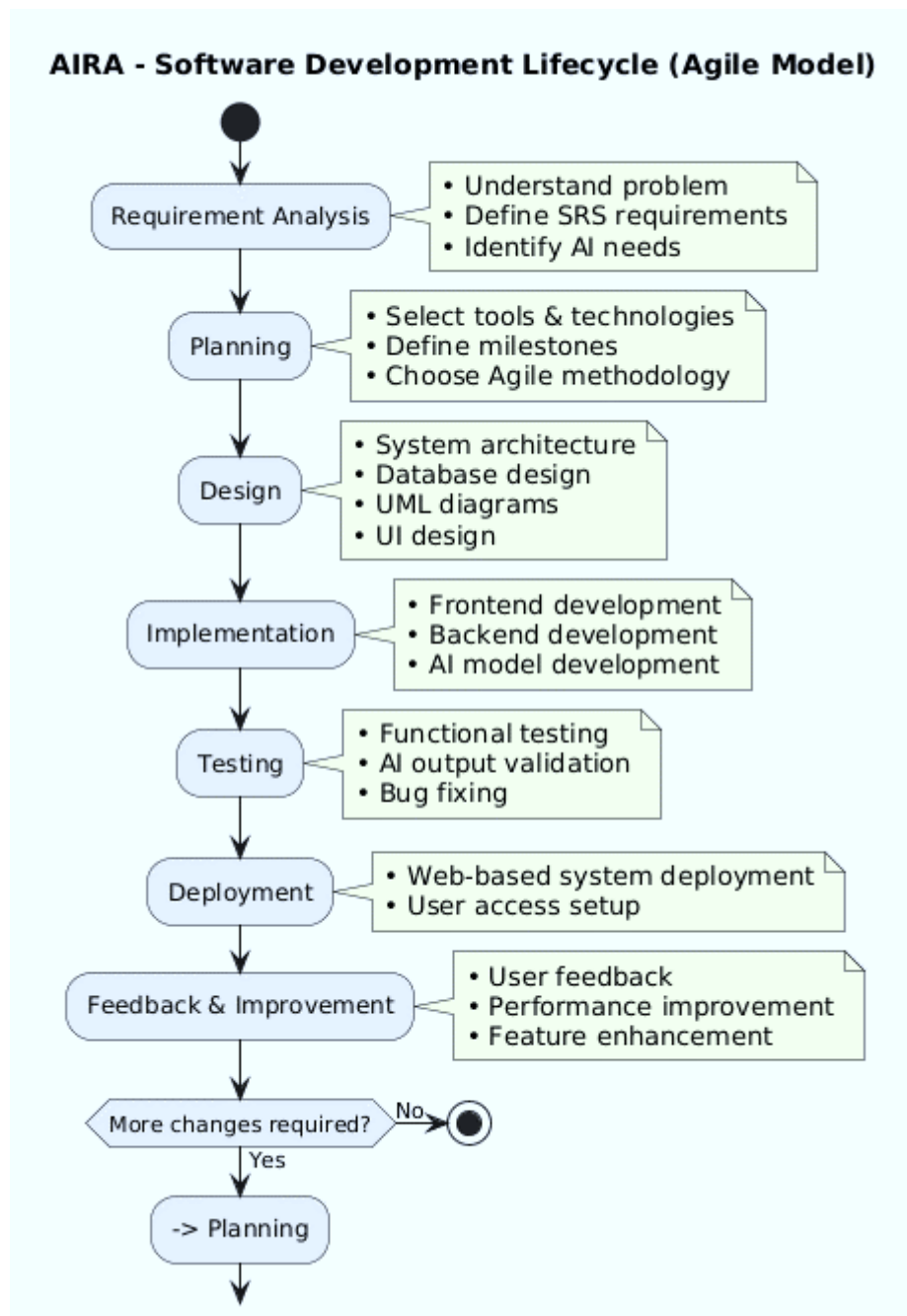


Figure 5: Software Lifecycle

3.5. Chapter Summary

This chapter described the updated software requirements according to the current implementation. It includes authentication, access limits, SRS generation, ambiguity checking, UML generation, UML image explanation, history, settings, and export workflows.

Chapter 4: Design and Architecture

4.1. Entity Relationship Diagram (ERD):

4.2. Sequence Diagrams:

4.3. Architecture Design Diagram:

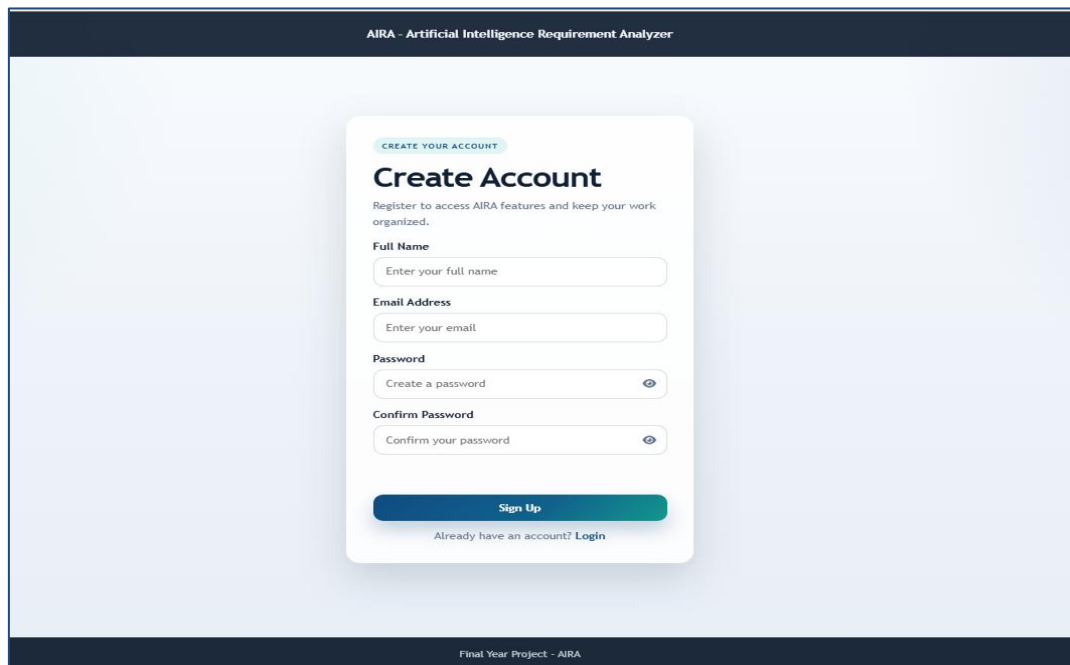
4.4. Class Diagram:

4.5. Database Design:

4.6. Interface Design:

The interface is implemented as static HTML pages styled with CSS and controlled by frontend JavaScript.

Sign Up Screen



The screenshot displays a web interface for the 'AIRA - Artificial Intelligence Requirement Analyzer'. At the top, a dark blue header contains the text 'AIRA - Artificial Intelligence Requirement Analyzer'. The main content area has a light blue background. Centered on the page is a white card with a light blue border. Inside the card, at the top, is a small teal button labeled 'CREATE YOUR ACCOUNT'. Below this is the heading 'Create Account' in bold. A subtext reads: 'Register to access AIRA features and keep your work organized.' The form consists of four input fields: 'Full Name' with the placeholder 'Enter your full name', 'Email Address' with 'Enter your email', 'Password' with 'Create a password' and a toggle icon, and 'Confirm Password' with 'Confirm your password' and a toggle icon. At the bottom of the card is a large teal 'Sign Up' button. Below the button, a link says 'Already have an account? Login'. The footer of the page is a dark blue bar with the text 'Final Year Project - AIRA'.

Figure 11: Sign Up Screen

Home Screen

Generate SRS Screen

The screenshot shows the 'SPECIFICATION BUILDER' section of the AIRA application. The main heading is 'Generate Software Requirement Specification'. Below this, a sub-heading reads: 'Start with a short idea, a reference document, or a UML diagram image. Add extra details when they are available.' The form includes a 'Project Topic' input field with the example text 'e.g. Weather Forecasting System'. Below this is an 'Upload Source File' section with the instruction 'Upload one UML image, PDF, Word document, text file, or supporting project file for this SRS.' and an 'Upload File' button. A toggle for 'Optional customization' is also present. At the bottom of the form is a 'Generate SRS' button. The footer of the application is visible, showing 'Final Year Project - AIRA'.

Figure 13: Screen SRS

~

Generate UML Diagram Screen

AIRA - Artificial Intelligence Requirement Analyzer

The screenshot shows the 'DIAGRAM WORKSPACE' section of the AIRA application. The main heading is 'Generate UML Diagram'. Below this, a sub-heading reads: 'Provide your system description or upload an SRS document. AIRA will generate UML diagram output and let you refine it visually.' The form includes a 'Select UML Diagram Type' dropdown menu with 'Use Case Diagram' selected. Below this is a 'System Description or Upload File' section with a text area for 'Write your system description, workflow, or modeling request here...'. There are two buttons: 'Attach File' and 'Attach Diagram to Edit'. At the bottom of the form are two buttons: 'Generate UML Diagram' and 'Open Editor'. The footer of the application is visible, showing 'Final Year Project - AIRA'.

Figure 14: UML Screen Diagram

UML Image Description Screen

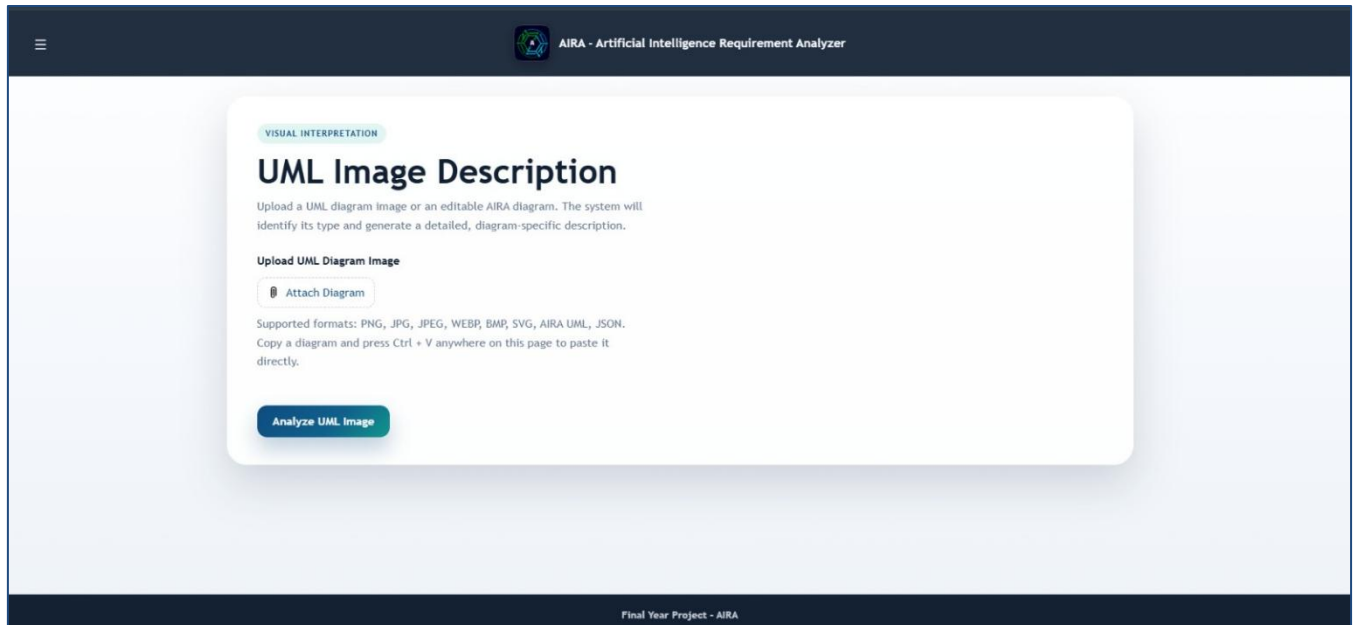


Figure 15: UML Image Description Screen

My History Screen

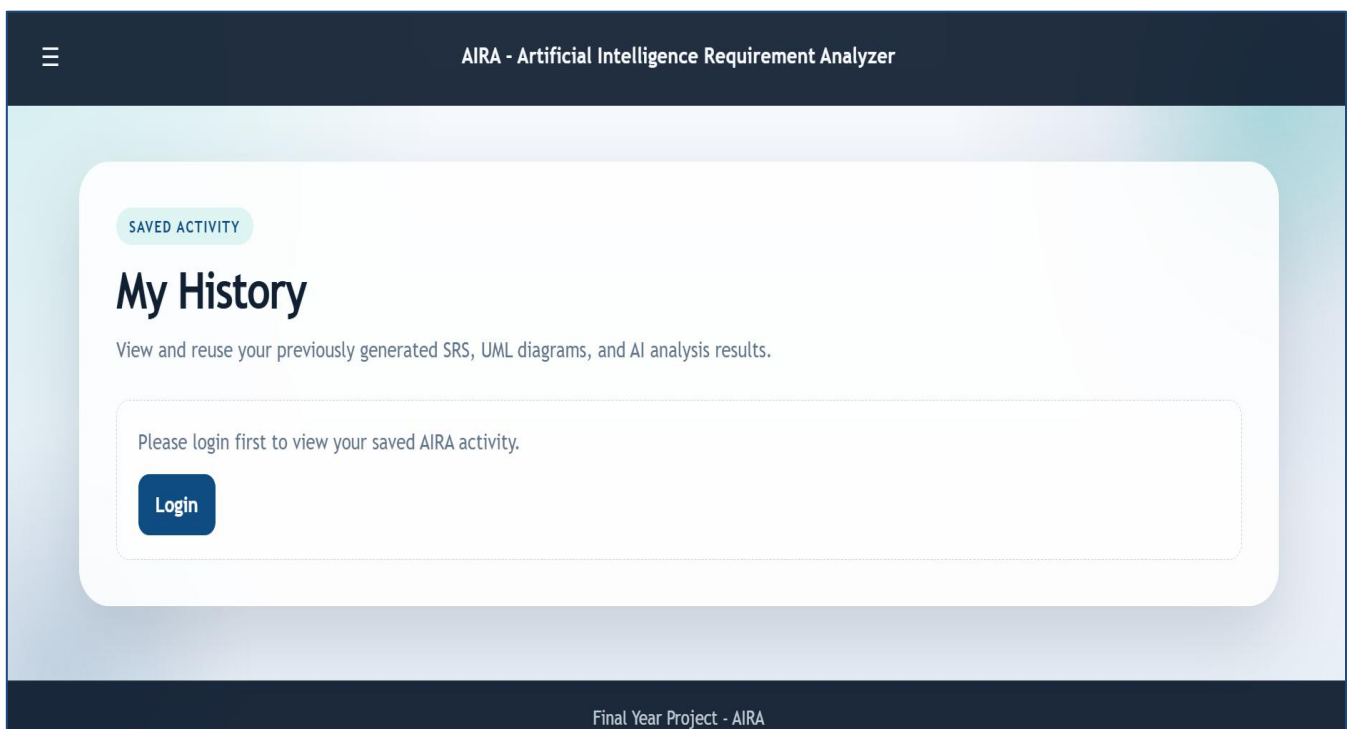


Figure 16: My History Screen

Setting

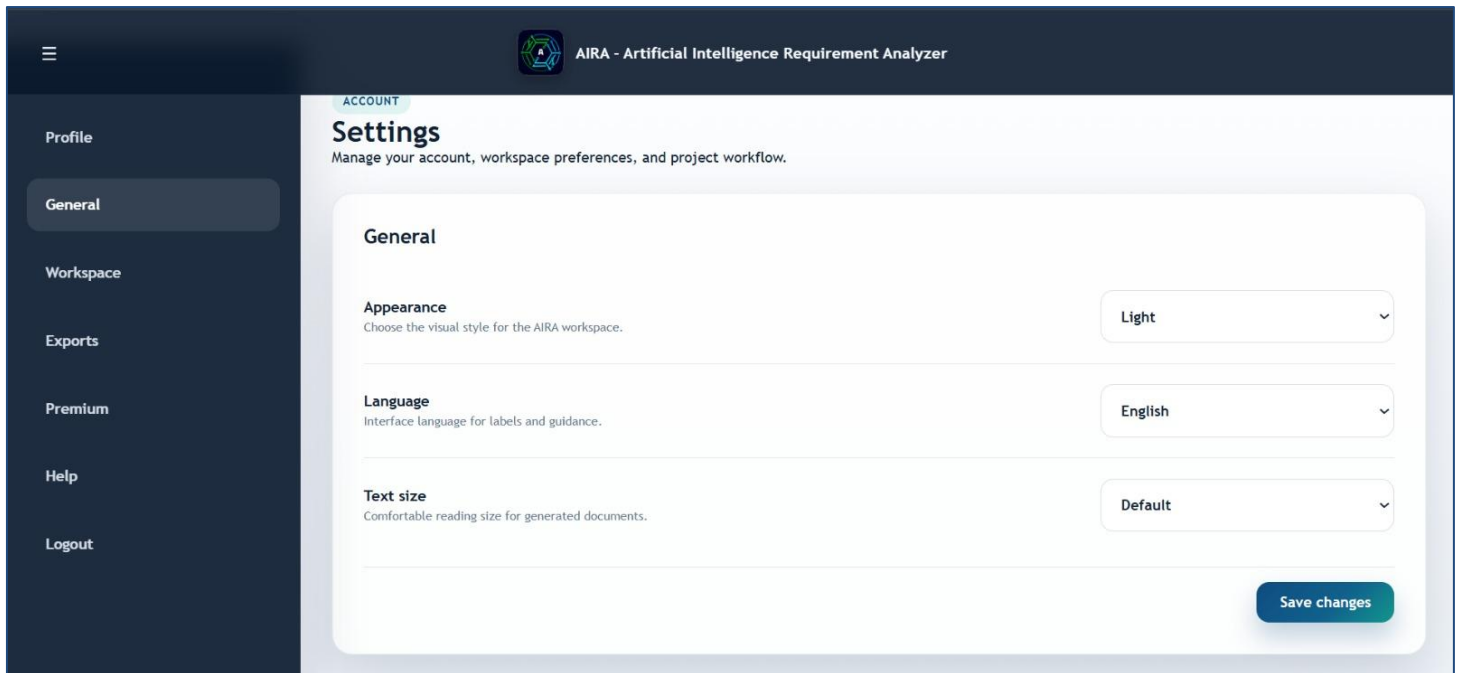


Figure 17: Setting

4.7. Summary

This chapter presented the updated design and architecture of AIRA using the current codebase as the source of truth. It described the ERD, sequences, architecture, module/class view, database design, and interface screens in the same report structure.

Chapter 5: Software Testing & Implementation

5.1 Deriving Test:

Tests for AIRA were derived directly from the implemented functional modules, backend routes, and AI service workflows. Since the project does not include a complete automated testing suite, all test cases are defined as **manual verification scenarios** for the final system execution.

Test derivation is based on mapping each major feature to its corresponding system component, including frontend interfaces, Express backend routes, MySQL database operations, and Python-based AI services.

The primary modules covered in testing include authentication, SRS generation, SRS ambiguity analysis, UML generation, UML image description, file extraction, and history management.

For SRS generation, test derivation focuses on validating multiple input sources such as prompts, structured fields, and uploaded documents, along with correct storage and retrieval of generated outputs. For UML generation, tests verify diagram type selection, PlantUML rendering, and editable workspace functionality.

OCR and AI-based features require special consideration because outputs depend on input quality, model behavior, and optional external API availability. Therefore, all AI-related outputs are treated as **draft results requiring manual verification** rather than absolute final outputs.

5.2 Test Environment:

The test environment should match the implemented local architecture: browser frontend, Express backend on localhost:3000, MySQL database named aira_db, Python runtime, model files, OCR tools, the system is tested in a local full-stack environment consisting of frontend, backend, database, Python service and optional cloud API keys.

Hardware:

- Intel Core i3 or higher
- Minimum 4GB RAM
- 256GB SSD or higher
- Internet connection (only required for optional cloud AI services and external rendering APIs)

Software:

- Windows 10 / Windows 11
- Google Chrome / Microsoft Edge / Mozilla Firefox
- Node.js with Express.js backend
- Python runtime environment
- MySQL database (aira_db)
- Visual Studio Code (development environment)
- scikit-learn, joblib, OpenCV, Tesseract OCR
- HTML5, CSS3, JavaScript frontend stack

5.3 Testing Techniques:

The following testing techniques are applied for system verification:

- **Functional Testing:**
Validates that all system features behave according to functional requirements.
- **Integration Testing:**
Ensures correct interaction between frontend, backend, database, and Python AI services.
- **Black Box Testing:**
Focuses on input-output validation without considering internal implementation logic.

Manual verification is used throughout the system due to the absence of a complete automated testing framework.

5.4 Test Cases:

Test Case ID: 1

Test Case Title: User Login

Field	Description
Test Case ID	TC-01
Test Case Title	User Login
Preconditions	Backend, MySQL database, and login system are running
Actions	Enter registered email and password, then click login
Expected Result	Valid credentials authenticate user and create session; invalid credentials return error
Actor	User
Result	Manual verification required

Table 4-1 User Login

Test Case ID: 2

Test Case Title: SRS Generation

Field	Description
Test Case ID	TC-02
Test Case Title	Generate SRS Document
Preconditions	User is logged in or guest fallback is enabled; Python SRS service and MySQL are running
Actions	Enter project details or upload supported file and generate SRS
Expected Result	Structured SRS is generated and stored in system/history
Actor	User
Result	Manual verification required

Table 4-2 SRS Generation

Test Case ID: 3**Test Case Title: UML Generation**

Field	Description
Test Case ID	TC-03
Test Case Title	Generate UML Diagram
Preconditions	Valid system description is provided
Actions	Select diagram type and generate UML output
Expected Result	PlantUML and editable diagram are generated (Use Case/Class/Sequence/ERD/Activity)
Actor	User
Result	Manual verification required

*Table 4-3 UML Generation***Test Case ID: 4****Test Case Title: SRS Ambiguity Analysis**

Field	Description
Test Case ID	TC-04
Test Case Title	Check SRS Ambiguity
Preconditions	Valid SRS text is available
Actions	Submit SRS for ambiguity analysis
Expected Result	System returns ambiguity report with detected issues and summary
Actor	User
Result	Manual verification required

*Table 4-4 SRS Ambiguity Analysis***5.5 Chapter Summary:**

This chapter presented the implementation-aware testing strategy for the AIRA system. Since no complete automated testing framework exists in the codebase, all test cases are defined as manual verification scenarios based on real system workflows.

The testing approach covers functional, integration, and black-box techniques to validate key modules including authentication, SRS generation, UML generation, ambiguity detection, file extraction, and OCR-based image analysis.

Overall, the system is verified through end-to-end manual execution, ensuring that major functionalities operate correctly within the local deployment environment, while all AI-generated outputs are treated as draft-level results requiring user validation.

.

Chapter 6: Conclusion and Future Work

6.1 Discussion:

AIRA is designed to reduce the effort involved in producing initial Software Requirements Specification (SRS) and UML documentation. The current implementation supports SRS generation, ambiguity detection, UML generation, UML image explanation, editing capabilities, export functions, access control, and history tracking.

Unlike traditional documentation tools that address isolated tasks, AIRA combines frontend interfaces, Express-based APIs, MySQL storage, Python-based AI processing, OCR functionality, trained local models, and optional integration with external AI providers into a unified system.

The system is intended to assist in drafting documentation rather than producing final verified artifacts. Generated outputs still require human review for correctness, completeness, and alignment with stakeholder requirements.

6.2 Conclusion:

AIRA presents an implemented AI-assisted system for software documentation support. It integrates natural language processing, machine learning components, OCR-based image interpretation, and diagram generation within a web-based environment backed by persistent database storage.

The system fulfills its primary objectives by providing modules for authentication, SRS generation, ambiguity analysis, UML generation, UML image description, history management, settings control, usage limitation, and PlantUML-based rendering.

The outputs generated by the system should be treated as structured drafts. Final academic or professional validation requires manual verification and review of both generated content and runtime environment configuration.

6.3 Limitations:

Despite its functionality, the current system has several limitations:

- AI-generated SRS and UML outputs may require manual correction and expert validation
- UML generation quality is dependent on input clarity and completeness
- OCR performance varies based on image quality, contrast, and diagram complexity
- External AI provider features depend on valid API keys and network availability
- The system is deployed as a local web application; production-grade cloud deployment is not implemented
- Formal regression, load, and end-to-end testing is not fully established in the current repository
- These constraints limit reliability in production environments and require further refinement before broader deployment.

6.4 Future Work:

Future development should focus on system validation, robustness, and deployment readiness:

- Implement automated unit, integration, end-to-end, and load testing across frontend, backend, database, and Python services
- Develop formal OpenAPI documentation for all backend endpoints and system interfaces
- Improve multilingual processing and evaluate system performance beyond English-language inputs
- Retrain and validate AI models using expanded datasets and external evaluation benchmarks
- Enhance OCR accuracy for low-quality, handwritten, and complex UML diagrams
- Introduce production-grade deployment with monitoring, backups, and security hardening
- Develop a complete administrative module for role management, usage tracking, and system control
- Improve system scalability for collaborative and multi-user environments

These improvements would strengthen system reliability and support transition from prototype to production-level deployment.

References

- [1] “ReqInOne: A Large Language Model-Based Agent for Software Requirements Specification Generation | IEEE Conference Publication | IEEE Xplore.” Accessed: Jun. 16, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11190373>
- [2] “Lucid Software Inc. Privacy Policy | Lucid.” Accessed: Jun. 16, 2026. [Online]. Available: <https://lucid.co/privacy>
- [3] “DiagramGPT – AI diagram generator created by Eraser.” Accessed: Jun. 16, 2026. [Online]. Available: <https://www.eraser.io/diagramgpt>
- [4] T. Zhu, L. C. Cordeiro, and Y. Sun, “ReqInOne: A Large Language Model-Based Agent for Software Requirements Specification Generation,” *Proceedings of the IEEE International Conference on Requirements Engineering*, pp. 449–457, 2025, doi: 10.1109/RE63999.2025.00054.
- [5] “AI App Builder — Create Apps with One Prompt | Taskade.” Accessed: Jun. 16, 2026. [Online]. Available: <https://www.taskade.com/create?spaceId=3qexce9rlqlg34yv>
- [6] “FREE Software Requirements Specification (SRS) Generator - PaceAI.” Accessed: Jun. 16, 2026. [Online]. Available: <https://paceai.co/tools/requirements-specification-generator/>
- [7] “NLTK :: Natural Language Toolkit.” Accessed: Jun. 16, 2026. [Online]. Available: https://www.nltk.org/?utm_source=chatgpt.com
- [8] “List | Project 1 | Laiba Arshad’s Workspace (List).” Accessed: Jun. 16, 2026. [Online]. Available: <https://app.clickup.com/90181789919/v/1/2kzm1u6z-438>
- [9] “2025 IEEE 33rd International Requirements Engineering Conference : RE 2025,” 2025.
- [10] “Documents.” Accessed: Jun. 16, 2026. [Online]. Available: https://lucid.app/documents#/home?folder_id=recent
- [11] “Documents.” Accessed: Jun. 16, 2026. [Online]. Available: https://lucid.app/documents#/home?folder_id=recent